

RFCs — POS Tech Challenge Fase 3

Request for Comments — decisões técnicas

RFC 0001 — Escolha do provedor de nuvem

- **Status:** Aceito
- **Data:** 2026-09-09
- **Autores:** Equipe POS Tech (13SOAT)
- **Repositórios afetados:** tech-challenge-database, tech-challenge-lambda-functions, tech-challenge-kubernetes, tech-challenge-application

Contexto

O Tech Challenge da Fase 3 exige elevar o sistema POS a um nível de operação corporativa: API Gateway, Function Serverless, banco de dados gerenciado, cluster Kubernetes escalável e infraestrutura como código, com escolha livre de nuvem. O sistema já vinha evoluindo com um deploy self-hosted em Minikube (repositório tech-challenge-application/infra); era necessário decidir onde a operação "real" (produção) do sistema passaria a rodar.

Motivadores da decisão

- Precisa oferecer, nativamente ou via serviço gerenciado equivalente: API Gateway, compute serverless (Function), banco de dados relacional gerenciado e Kubernetes gerenciado (ou cluster próprio via IaC).
- Terraform como ferramenta de provisionamento obrigatória — o provedor precisa ter um provider Terraform maduro.
- Menor custo/curva de aprendizado possível para uma equipe pequena entregando em prazo de curso.
- Reaproveitar o máximo de conhecimento e documentação já produzidos (grande parte do ecossistema de Function Serverless madura em torno de AWS Lambda).

Opções consideradas

Opção	Prós	Contras
AWS	API Gateway, Lambda, RDS, EKS todos maduros e com provider Terraform de primeira classe; maior volume de documentação/exemplos disponíveis para o time	Curva de custo em serviços gerenciados (RDS, EKS) se não desligados
Azure	App Gateway, Functions, Azure SQL, AKS equivalentes	Menor familiaridade da equipe; provider Terraform menos usado no material de referência do curso
GCP	API Gateway, Cloud Functions, Cloud SQL, GKE equivalentes	Mesmo ponto de familiaridade; ecosystem de exemplos mais escasso no contexto do curso

Decisão

Adotar **AWS** como provedor de nuvem único para toda a infraestrutura de produção do sistema POS:

- **API Gateway** → AWS API Gateway (HTTP API), provisionado em `tech-challenge-lambda-functions/apigateway.tf`.
- **Function Serverless** → AWS Lambda (Node.js 24), mesmo repositório.
- **Banco de dados gerenciado** → Amazon RDS (MySQL 8.0), provisionado em `tech-challenge-database`.
- **Cluster Kubernetes** → Amazon EKS, provisionado em `tech-challenge-kubernetes`.
- **IaC** → Terraform com provider hashicorp/aws em todos os repositórios de infraestrutura.

O ambiente Minikube self-hosted (`tech-challenge-application/infra`) é mantido apenas como ambiente de desenvolvimento/homologação local, não como produção.

Consequências

- Todos os repositórios de infraestrutura compartilham a mesma conta/região AWS (`us-east-1`), permitindo referenciar recursos entre eles via Terraform `data source` (ex.: `tech-challenge-lambda-functions` lê a VPC/RDS de `tech-challenge-database`).
- Um único conjunto de credenciais AWS (via GitHub Actions secrets) precisa ser gerenciado por repositório de infraestrutura.
- Vendor lock-in em AWS é aceito conscientemente — não há requisito de portabilidade multi-nuvem neste desafio.
- Custos reais de RDS/EKS passam a existir enquanto os recursos estiverem provisionados (mitigado por serem `db.t3.micro` / `node group t3.small` de 1–2 nós).

RFC 0002 — Escolha do banco de dados gerenciado

- **Status:** Aceito
- **Data:** 2026-09-09
- **Autores:** Equipe POS Tech (13SOAT)
- **Repositórios afetados:** tech-challenge-database, tech-challenge-application, tech-challenge-lambda-functions, tech-challenge-kubernetes

Contexto

O sistema já usava MySQL 8.0 desde as fases anteriores (via Eloquent ORM na aplicação Laravel, rodando em container local/Minikube). Para a Fase 3, o desafio pede um banco de dados **gerenciado** (RDS, Cloud SQL ou equivalente), compartilhado entre a aplicação principal e a nova Function Serverless de autenticação — ambas precisam ler os mesmos dados de cliente.

Motivadores da decisão

- Zero fricção de migração de schema/queries já escritas contra MySQL 8.0 (DDL, AUTO_INCREMENT, tipos, todas as migrations Laravel existentes).
- Precisa ser acessível tanto pela aplicação Laravel (rodando em EKS) quanto pela Function Serverless (Node.js/Lambda) — ambas dentro da mesma VPC.
- Gerenciado pela nuvem (backups automáticos, patching, alta disponibilidade configurável), eliminando a necessidade de manter um MySQL em container como fonte de verdade de produção.
- Compatibilidade com php-open-source-saver/jwt-auth e bibliotecas Node (mysql2/equivalente) já usadas nos protótipos do Lambda.

Opções consideradas

Opção	Prós	Contras
Amazon RDS para MySQL 8.0	Mesma engine já usada nas fases anteriores — nenhuma migration/queries precisam mudar; mesma sintaxe entre app e Lambda	Nenhum contra relevante dado o contexto — a troca de engine traria custo de migração sem benefício correspondente
Amazon RDS para PostgreSQL	Mais rico em tipos/extensões	Exigiria reescrever migrations e revisar compatibilidade de tipos com o Eloquent já implementado — sem ganho justificável neste desafio
Aurora Serverless v2 (MySQL-compatible)	Escala automaticamente a zero, custo potencialmente menor em baixa carga	Complexidade adicional (capacity units, cold start) desnecessária para o volume de uma oficina; equipe sem experiência prévia com Aurora
Manter MySQL em container	Zero mudança de	Não atende ao requisito

(sem gerenciado)	infraestrutura	obrigatório do desafio (banco de dados gerenciado); sem backups/HA nativos
------------------	----------------	--

Decisão

Adotar **Amazon RDS para MySQL 8.0.46** (`aws_db_instance`, classe `db.t3.micro`) como banco de dados gerenciado único, compartilhado entre `tech-challenge-application` (via EKS) e `tech-challenge-lambda-functions` (via Lambda), provisionado em `tech-challenge-database/rds.tf` com `identifier` fixo `techchallenge-rds`.

O ambiente Minikube local mantém seu próprio MySQL em cluster, isolado — não é o mesmo banco, apenas um ambiente de desenvolvimento.

Justificativa formal e ajustes no modelo relacional

O modelo relacional em si **não muda** de engine nem de estrutura ao migrar de MySQL em container para RDS MySQL — é a mesma engine, então todas as migrations Laravel existentes (`Customer`, `Vehicle`, `ServiceOrder`, `Item`, `Service`, `Notification`, `Stock`, etc...), continuam válidas sem alteração de DDL.

Os ajustes necessários foram exclusivamente de **conectividade e identidade do recurso**, não de modelagem:

- `Identifier` fixo (`techchallenge-rds`) para permitir que `tech-challenge-lambda-functions` localize a instância via data `"aws_db_instance"` sem `hardcode` de endpoint.
- Regra de ingress liberando a porta 3306 para toda a VPC (`aws_vpc_security_group_ingress_rule.rds_mysql_from_vpc`), já que tanto os nós EKS quanto a ENI da Lambda residem na mesma VPC mas em subnets/security groups distintos.
- Os relacionamentos entre entidades (`Customer 1—N Vehicle`, `ServiceOrder N—N Service/Item`, etc.) são os mesmos documentados nos diagramas ER das fases anteriores — nenhuma mudança de cardinalidade foi necessária para o modelo gerenciado.

Consequências

- Um único ponto de falha de dados (a instância RDS) passa a ser compartilhado por dois sistemas (app e Lambda) — reforça a necessidade de backups automáticos (habilitados por padrão no RDS) e de monitoramento de conexões simultâneas.
- Credenciais (`db_password`) precisam ser mantidas idênticas entre `tech-challenge-database`, `tech-challenge-kubernetes` (`ConfigMap/Secret`) e `tech-challenge-lambda-functions` (`Secrets Manager`) — divergência quebra silenciosamente a autenticação (ver `tech-challenge-lambda-functions/README.md`).
- `publicly_accessible = true` na instância atual é um ponto de atenção de segurança para revisão futura (deveria ser `false`, com acesso só via VPC/bastion), registrado como pendência conhecida e não tratado neste RFC.

RFC 0003 — Estratégia de autenticação (API Gateway + Function Serverless + CPF)

- **Status:** Aceito
- **Data:** 2026-09-09
- **Autores:** Equipe POS Tech (13SOAT)
- **Repositórios afetados:** tech-challenge-lambda-functions, tech-challenge-application, tech-challenge-kubernetes

Contexto

A aplicação já usava JWT via `php-open-source-saver/jwt-auth` para proteger rotas internas (`middleware('auth:api')`). Nesta Fase, exige que o *ponto de entrada* de autenticação passe a ser um API Gateway com uma Function Serverless dedicada, que valide o CPF do cliente, consulte sua existência/status no banco, e emita um token JWT válido para consumo das APIs protegidas — sem depender do monólito Laravel para esse primeiro passo.

Motivadores da decisão

- Requisito obrigatório do desafio: autenticação via CPF, atrás de um API Gateway, resolvida por uma Function Serverless.
- O JWT emitido precisa ser aceito pelo `guard auth:api` já existente na aplicação Laravel — ou seja, dois sistemas de tecnologias diferentes (Node.js e PHP) precisam assinar/validar tokens de forma compatível.
- Evitar duplicar a lógica de autenticação dentro do monólito; a Function Serverless deve ser a única emissora de tokens para o fluxo de login.

Opções consideradas

Opção	Prós	Contras
Function Serverless (Lambda) emite JWT compatível com o guard Laravel	Desacopla o login do deploy do monólito; escala independentemente; atende ao requisito de "Function Serverless" ao pé da letra	Exige manter <code>`jwt_secret`</code> idêntico entre Lambda e Laravel (acoplamento de configuração, não de código)
Delegar login para uma rota Laravel por trás do API Gateway (sem Lambda real)	Reaproveita 100% do código de autenticação já existente	Não atende ao requisito obrigatório de Function Serverless dedicada
AWS Cognito com fluxo customizado para CPF	Serviço gerenciado de identidade, menos código para manter	CPF como identificador primário não é um caso nativo do Cognito; exigiria triggers Lambda de qualquer forma, sem reduzir a complexidade real

Decisão

- **API Gateway** (AWS API Gateway HTTP API) expõe POST /login, roteando para a **Function Serverless** (tech-challenge-lambda-functions, Node.js 24).
- A função:
 1. Valida o identificador recebido (CPF ou e-mail) e a senha.
 2. Consulta a existência e o status do cliente diretamente na RDS compartilhada (mesma instância descrita na RFC 0002 (0002-escolha-do-banco-de-dados.md)).
 3. Gera e devolve um JWT (access_token, token_type: bearer) assinado com o mesmo jwt_secret configurado como JWT_SECRET na aplicação Laravel (tech-challenge-kubernetes GitHub secret) — garantindo que o guard auth:api do monólito aceite o token sem nenhuma alteração de código.
- O segredo de assinatura (jwt_secret) e as credenciais de banco (db_password) são armazenados no **AWS Secrets Manager** (tech-challenge-lambda-functions/secrets.tf), não em variáveis de ambiente em texto plano na função.
- Rotas sensíveis da aplicação (/customer, /vehicle, /service-order, etc.) continuam protegidas pelo middleware auth:api do Laravel, que agora aceita tanto tokens emitidos pelo fluxo interno (/auth/login) quanto pelo novo fluxo via Lambda — ambos compatíveis porque compartilham o mesmo segredo de assinatura.

Consequências

- **Acoplamento de configuração explícito:** jwt_secret, db_name, db_user e db_password precisam permanecer idênticos entre os três repositórios (tech-challenge-lambda-functions, tech-challenge-kubernetes, tech-challenge-database) — documentado em detalhe no README.md do repositório do Lambda. Divergência causa falha silenciosa de autenticação (o login "não encontra" o usuário ou o token é rejeitado).
- Dois pontos de emissão de JWT existem no sistema (o fluxo legado /auth/login do Laravel e o novo fluxo via Lambda) — ambos válidos e mutuamente compatíveis por design, não depreciados um pelo outro nesta fase.
- Testes unitários do Lambda mockam banco e Secrets Manager (não exigem AWS real), mantendo o mesmo padrão de isolamento de dependências já usado nos testes de UseCase da aplicação Laravel (Mockery).
- Rotação do jwt_secret no futuro precisa ser coordenada nos dois repositórios simultaneamente — não há hoje um mecanismo automatizado de propagação, apenas o secret manual em cada CI/CD.